

1.3 Numpy与线性代数

1.3.1 基本线性代数运算

1. 逆矩阵

首先我们来看看矩阵的求逆。

逆矩阵是线性代数中的一个基本概念，主要用于解线性方程组、矩阵的分解以及计算机图形学等领域。对于一个给定的方阵（即行数和列数相等的矩阵） A ，如果存在另一个方阵 B ，使得 $AB = BA = I$ （其中 I 是单位矩阵，即对角线上元素为1，其余元素为0的矩阵），则称 B 为 A 的逆矩阵，记作 A^{-1} 。

逆矩阵的求法可以用很多方法求，其中比较典型的包括：

1. 使用高斯-约旦消元法（Gauss-Jordan Elimination）：

高斯-约旦消元法是一种将矩阵通过行变换化为单位矩阵的算法，其过程中可以同时求得原矩阵的逆矩阵（如果存在的话）。这种方法的基本思想是通过行变换将矩阵转换为单位矩阵，同时记录下这些行变换，这些变换的逆变换即为原矩阵的逆矩阵。

2. 使用伴随矩阵与行列式：

如果矩阵 A 的行列式 $|A| \neq 0$ （即矩阵是可逆的），则 A 的逆矩阵可以通过其伴随矩阵 $adj(A)$ 与行列式 $|A|$ 的倒数来计算： $A^{-1} = \frac{1}{|A|} adj(A)$ 。伴随矩阵 $adj(A)$ 是矩阵 A 的代数余子式矩阵的转置。

在Python中，NumPy库提供了非常方便的函数来计算矩阵的逆。如果你已经安装了NumPy，可以使用 `numpy.linalg.inv` 函数来计算逆矩阵。以下是一个简单的示例：

```
import numpy as np

# 定义一个方阵A
A = np.array([[1, 2], [3, 4]])

# 计算A的逆矩阵
A_inv = np.linalg.inv(A)

print("矩阵A:")
print(A)
print("矩阵A的逆矩阵:")
print(A_inv)

# 验证A * A_inv 是否等于单位矩阵
I = np.dot(A, A_inv)
print("A * A_inv =")
print(I)
```

注意，如果矩阵 A 不可逆（即其行列式为0），`np.linalg.inv(A)` 将抛出一个 `LinAlgError` 错误。因此，在实际应用中，你可能需要先检查矩阵是否可逆，例如通过计算其行列式：

```
if np.linalg.det(A) != 0:
    A_inv = np.linalg.inv(A)
else:
    print("矩阵A不可逆")
```

而伪逆矩阵的出现，主要是为了解决在线性代数中遇到的一些问题，特别是在处理非方阵（即行数和列数不相等的矩阵）或奇异矩阵（即行列式为0的方阵）时，这些矩阵没有传统意义上的逆矩阵。伪逆矩阵提供了一种在这些情况下求解线性方程组或最小化二乘误差的方法。

在NumPy库中，`np.linalg.pinv` 函数用于计算矩阵的伪逆。这个函数可以处理非方阵和奇异矩阵，并返回它们的伪逆矩阵。

```
import numpy as np

# 定义一个非方阵或奇异方阵A
A = np.array([[1, 2, 3], [4, 5, 6]]) # 非方阵示例

# 计算A的伪逆矩阵
A_pinv = np.linalg.pinv(A)

print("矩阵A:")
print(A)
print("矩阵A的伪逆矩阵:")
print(A_pinv)

# 如果A是方阵且非奇异，A_pinv将接近于A的逆矩阵（但不是完全相同，因为计算方法和数值稳定性差异）
# 如果A是非方阵或奇异方阵，A_pinv将提供一个在某种意义上“最优”的逆矩阵
```

注意事项：

- 伪逆矩阵不是唯一的，但 `np.linalg.pinv` 函数通常会返回一个满足特定条件的伪逆矩阵（如最小化二范数解）。
- 当矩阵A是方阵且非奇异时，`np.linalg.pinv(A)` 的结果将接近于但通常不完全等于 `np.linalg.inv(A)`，因为计算伪逆的算法和计算逆矩阵的算法在数值上可能存在微小差异。
- 在使用伪逆矩阵时，应注意其应用场景和限制条件，以确保得到的结果符合实际需求。

2. 矩阵的秩

矩阵的秩是线性代数中的一个重要概念，它表示矩阵中行或列之间的线性关系。具体来说，矩阵的秩定义为该矩阵所有子式中非零子式的最高阶数。换句话说，如果我们从矩阵中挑选出一些行和列，组成一个子式，那么这个子式的阶数最大能是多少，这个数就是矩阵的秩。在另一种等价定义中，矩阵的秩也可以理解为矩阵中独立行（或列）的数量，即矩阵的行向量组（或列向量组）的秩。

矩阵的秩有很多重要的性质和应用，比如它可以用于判断矩阵是否可逆（非零方阵的秩等于其阶数时，矩阵可逆），以及确定线性方程组解的存在性和唯一性（如果系数矩阵的秩小于增广矩阵的秩，则方程组无解；如果系数矩阵的秩等于方程个数，则方程组有唯一解；如果系数矩阵的秩小于方程个数，则方程组有无穷多解）。

在Python中，NumPy库提供了非常方便的函数来计算矩阵的秩。具体来说，可以使用 `numpy.linalg.matrix_rank` 函数来计算矩阵的秩。以下是一个使用NumPy求矩阵秩的示例：

```
import numpy as np

# 定义一个矩阵
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# 计算矩阵的秩
rank = np.linalg.matrix_rank(matrix)

print("矩阵的秩为:", rank)
```

在这个示例中，我们首先导入了NumPy库，并定义了一个3x3的矩阵。然后，我们使用 `np.linalg.matrix_rank` 函数来计算这个矩阵的秩，并将结果打印出来。注意，虽然这个矩阵的所有元素都是连续的整数，但根据矩阵秩的定义，它的秩并不等于3，而是小于3（在这个具体例子中，秩为2，因为矩阵的第三行是前两行的线性组合）。

3. 行列式

行列式是一个数学概念，用于描述一个矩阵的性质。具体来说，行列式是一个方阵中所有元素按照一定规则排列后所得到的一个标量值。它反映了矩阵的某种固有属性，如矩阵是否可逆、是否存在非零向量被映射到零向量等。

行列式的计算公式对于不同大小的矩阵有所不同。以下是一些常见的公式：

- 对于2x2矩阵A，其行列式|A|的公式为：

$$|A| = a_{11}a_{22} - a_{12}a_{21}$$

其中， a_{ij} 表示矩阵A中第i行第j列的元素。

- 对于3x3矩阵A，其行列式|A|的公式更为复杂，通常使用拉普拉斯展开或代数余子式的方法来计算。一个简化的公式（按第一行展开）为：

$$|A| = a_{11}(a_{22}a_{33} - a_{23}a_{32}) - a_{12}(a_{21}a_{33} - a_{23}a_{31}) + a_{13}(a_{21}a_{32} - a_{22}a_{31})$$

对于更大的矩阵，行列式的计算通常需要使用递归方法或特定的数学软件来辅助。

在Python中，NumPy库提供了 `numpy.linalg.det()` 函数来计算矩阵的行列式。以下是一个具体的示例：

```
import numpy as np

# 创建一个3x3的矩阵
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 0]])

# 计算矩阵的行列式
det = np.linalg.det(matrix)

# 输出结果
print("矩阵:\n", matrix)
print("行列式:", det)
```

在这个示例中，我们首先创建了一个3x3的矩阵 `matrix`，然后使用 `np.linalg.det(matrix)` 计算了它的行列式，并将结果存储在变量 `det` 中。最后，我们输出了矩阵和它的行列式值。需要注意的是，由于浮点数的精度问题，计算得到的行列式值可能会略有不同。如果需要高精度的计算，可以考虑使用其他数学软件或库。

4. 范数

矩阵范数用于衡量矩阵的“大小”或“规模”。它是一个将矩阵映射到非负实数的函数，满足一系列特定的性质，如非负性、齐次性、三角不等式等。矩阵范数有多种定义方式，常见的有L1范数、L2范数（也称为弗罗贝尼乌斯范数）、无穷范数等。

以弗罗贝尼乌斯范数为例，对于矩阵 $A \in \mathbb{R}^{m \times n}$ ，其弗罗贝尼乌斯范数定义为：

$$\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2}$$

其中， a_{ij} 是矩阵A中第i行第j列的元素。这个范数实际上是矩阵元素平方和的平方根，类似于向量的欧几里得范数。

范数和行列式的异同点

相同点：

- 它们都是矩阵的某种“度量”，用于描述矩阵的某种性质或大小。

不同点：

- **定义和性质**：行列式是一个标量值，与矩阵的行列变换和逆矩阵等性质紧密相关；而矩阵范数是一个将矩阵映射到非负实数的函数，满足非负性、齐次性和三角不等式等性质。
- **用途**：行列式主要用于判断矩阵是否可逆、求解线性方程组、计算矩阵的特征多项式等；而矩阵范数则更多地用于矩阵的数值分析、稳定性分析、条件数计算等领域。
- **计算复杂度**：一般来说，计算行列式的复杂度相对较低（特别是对于小型矩阵或特殊类型的矩阵）；而计算矩阵范数的复杂度可能较高，特别是对于大型矩阵或需要高精度计算的场景。

在NumPy中，可以使用 `numpy.linalg.norm()` 函数来计算矩阵的范数。这个函数非常灵活，可以通过设置不同的参数来计算不同类型的范数。

以下是一个使用NumPy计算矩阵弗罗贝尼乌斯范数的示例：

```
import numpy as np

# 定义一个矩阵
A = np.array([[1, 2], [3, 4]])

# 计算矩阵的弗罗贝尼乌斯范数（默认参数即为弗罗贝尼乌斯范数）
norm_A = np.linalg.norm(A)

# 或者显式指定范数为'fro'
norm_A_fro = np.linalg.norm(A, 'fro')

# 输出结果
print("矩阵A的弗罗贝尼乌斯范数为:", norm_A)
print("矩阵A的弗罗贝尼乌斯范数（显式指定）为:", norm_A_fro)

# 如果想要计算其他类型的范数（如L1范数或无穷范数），可以修改norm函数的参数
norm_A_1 = np.linalg.norm(A, 1) # 计算L1范数（列和范数）
norm_A_inf = np.linalg.norm(A, np.inf) # 计算无穷范数（行和范数）

print("矩阵A的L1范数为:", norm_A_1)
print("矩阵A的无穷范数为:", norm_A_inf)
```

在这个例子中，我们首先定义了一个2x2的矩阵 A，然后使用 `numpy.linalg.norm()` 函数计算了它的弗罗贝尼乌斯范数、L1范数和无穷范数，并将结果打印出来。注意，`numpy.linalg.norm()` 函数的第一个参数是待计算范数的矩阵，第二个参数是范数的类型（默认为弗罗贝尼乌斯范数）。

1.3.2 线性方程组的求解

1. 利用克拉默法则

克拉默法则是线性代数中一个用于求解线性方程组（方程组中方程的个数与未知数的个数相等，且系数矩阵的行列式不为零）的定理。它表述为：如果线性方程组

$$Ax = b$$

的系数矩阵 A 是一个 $n \times n$ 矩阵，且 A 的行列式 $|A| \neq 0$ ，则方程组有唯一解，且解的第 i 个分量 x_i 可以用行列式表示为：

$$x_i = \frac{\det(A_i)}{\det(A)}$$

其中， A_i 是将 A 的第 i 列替换为向量 b 后得到的矩阵。

由于NumPy库提供了计算行列式的 `numpy.linalg.det()` 函数，我们可以利用这个函数来实现克拉默法则。

```

import numpy as np

def cramer_rule(A, b):
    """
    使用克拉默法则求解线性方程组  $Ax = b$ 
    :param A: 系数矩阵，形状为 (n, n)
    :param b: 常数项向量，形状为 (n,)
    :return: 方程组的解，形状为 (n,)
    """
    n = A.shape[0] # 方程组的未知数个数
    if A.shape[1] != n or b.shape[0] != n:
        raise ValueError("系数矩阵A和常数项向量b的维度不匹配")

    det_A = np.linalg.det(A) # 计算系数矩阵A的行列式
    if det_A == 0:
        raise ValueError("系数矩阵A的行列式为零，方程组无解或有无穷多解")

    x = np.zeros(n) # 初始化解向量
    for i in range(n):
        # 构造矩阵Ai，即将A的第i列替换为b
        Ai = A.copy()
        Ai[:, i] = b
        # 计算Ai的行列式并除以A的行列式得到解的第i个分量
        x[i] = np.linalg.det(Ai) / det_A

    return x

# 示例
A = np.array([[4, 3, 2], [2, -2, 4], [3, 8, 2]])
b = np.array([1, 5, 6])

solution = cramer_rule(A, b)
print("方程组的解为:", solution)

```

在这个程序中，我们首先检查系数矩阵 A 和常数项向量 b 的维度是否匹配，然后计算 A 的行列式。如果行列式为零，则抛出异常，因为此时方程组可能无解或有无穷多解。接下来，我们遍历每个未知数，构造相应的 A_i 矩阵，并计算其行列式，最后通过克拉默法则的公式求解出每个未知数的值。最后，返回解向量 x 。

2. 高斯消元法

高斯消元法是一种用于解线性方程组的算法，它通过对方程组的系数矩阵进行行变换（包括乘以非零常数、行交换和行相加），将其转化为上三角矩阵形式，进而简化求解过程。当系数矩阵转化为上三角矩阵后，可以逐行求解出每个未知数的值。

高斯消元法通常包括两个主要阶段：前向消元（也称为部分选主元高斯消元法）和回代过程。前向消元用于将矩阵转化为上三角形式，而回代过程则用于从上到下求解每个未知数的值。

我们可以手动实现高斯消元法以更好地理解其工作原理。以下是一个使用NumPy实现高斯消元法的简单程序：

```

import numpy as np

def gaussian_elimination(A, b):
    """
    使用高斯消元法求解线性方程组  $Ax = b$ 
    :param A: 系数矩阵，形状为 (n, n)
    :param b: 常数项向量，形状为 (n,)
    :return: 方程组的解，形状为 (n,)
    """
    n = A.shape[0]
    if A.shape[1] != n or b.shape[0] != n:
        raise ValueError("系数矩阵A和常数项向量b的维度不匹配")

    # 合并A和b以简化操作
    Ab = np.hstack((A, b.reshape(-1, 1)))

    # 前向消元过程
    for k in range(n):
        # 选择主元（这里简单起见，不进行部分选主元，实际应用中可能需要）
        # 如果当前行的主元为0，则应该进行行交换（这里省略了行交换的逻辑）

        # 将当前行以下的所有行减去当前行的适当倍数，使得以下行的第k列变为0
        for i in range(k+1, n):
            factor = Ab[i, k] / Ab[k, k]
            Ab[i, k:] -= factor * Ab[k, k:]

    # 回代过程
    x = np.zeros(n)
    for i in range(n-1, -1, -1):
        x[i] = (Ab[i, -1] - np.dot(Ab[i, :i], x[:i])) / Ab[i, i]

    return x

# 示例
A = np.array([[2, 1, -1], [-3, -1, 2], [-2, 1, 2]])
b = np.array([8, -11, -3])

solution = gaussian_elimination(A, b)
print("方程组的解为:", solution)

```

在这个程序中，我们首先检查系数矩阵 A 和常数项向量 b 的维度是否匹配。然后，我们创建一个新的矩阵 Ab ，将 A 和 b 合并成一个增广矩阵，以便在消元过程中同时处理系数和常数项。接下来，我们进行

前向消元过程，将矩阵转化为上三角形式。最后，我们通过回代过程从上到下求解出每个未知数的值，并返回解向量 x 。

需要注意的是，这个实现为了简化起见，没有包含部分选主元的逻辑，也没有处理主元为0（即需要行交换）的情况。在实际应用中，这些情况都需要妥善处理以确保算法的鲁棒性。此外，对于大型稀疏矩阵，可能还需要使用更高效的稀疏矩阵存储和运算技术来优化算法性能。

3. `numpy.linalg.solve` 函数

`numpy.linalg.solve` 函数用于求解线性方程组或矩阵方程。给定一个方阵 a 和一个一维数组 b ，`solve` 函数会找到 x ，使得 $ax = b$ 。这里， a 的维度是 (n, n) ， b 的维度是 $(n,)$ ，解 x 的维度也将是 $(n,)$ 。

用法：

```
numpy.linalg.solve(a, b)
```

- **参数：**

- a ：系数矩阵，一个 (n, n) 的二维数组或矩阵。
- b ：常数项向量，一个 $(n,)$ 形状的一维数组或矩阵。

- **返回值：**

- 返回一个一维数组，表示方程组的解。

注意：

- a 必须是方阵且可逆（即行列式不为零）。
- 如果 a 不是方阵或不可逆，函数将抛出 `LinAlgError` 异常。

`numpy.linalg.lstsq` 函数用于求解最小二乘问题。当方程的数量多于未知数的数量（即，超定系统）时，`lstsq` 可以找到使残差的平方和最小的解。此外，它也可以用于求解恰好确定或欠定的系统。

目的：

- 求解形如 $Ax = b$ 的线性方程组的最小二乘解，其中 A 的列可能多于行数，即存在多于一个可能的解，但我们需要找到使残差平方和最小的那个解。

用法：

```
numpy.linalg.lstsq(a, b, rcond=None)[0]
```

- **参数：**

- a ：系数矩阵，一个 (m, n) 的二维数组或矩阵，其中 m 是方程的数量， n 是未知数的数量。
- b ：常数项向量或矩阵，形状为 $(m,)$ 或 (m, k) ，其中 k 是方程组的数量（如果同时求解多个方程组）。
- $rcond$ ：用于确定有效秩的截止条件。如果 $rcond$ 为 `None`，则使用机器精度的估计值。

- **返回值：**

- 返回一个元组，其中包含最小二乘解、残差、秩、奇异值和条件数。通常，我们只对第一个元素（最小二乘解）感兴趣。

注意：

- `lstsq` 函数返回的是所有可能解中残差平方和最小的那一个。
- 如果 A 是方阵且可逆，则 `lstsq` 的解与 `solve` 的解相同（在数值误差范围内）。
- 对于欠定系统（未知数多于方程），`lstsq` 会找到使残差平方和最小的解之一，这通常不是唯一的。

示例：

```
import numpy as np

# 系数矩阵A
A = np.array([[1, 2], [3, 4], [5, 6]])
# 常数项向量b
b = np.array([1, 2, 3])

# 使用lstsq求解最小二乘解
x, residuals, rank, s = np.linalg.lstsq(A, b, rcond=None)
print("最小二乘解:", x)
```

在这个例子中，由于 A 的行数多于列数，因此方程组是超定的，`lstsq` 会找到使残差平方和最小的解。

1.3.3 矩阵分解

1. 特征值分解

矩阵的特征值分解（Eigenvalue Decomposition, EVD）是一种将矩阵分解为其特征值和特征向量表示的矩阵之积的方法，也称为谱分解（Spectral Decomposition）。这种方法在数值计算、机器学习、图像处理、计算机视觉等领域有着广泛的应用。

设 A 是一个 $n \times n$ 的矩阵， λ 是一个数，若存在 n 维非零向量 x ，使得 $Ax = \lambda x$ ，则称 λ 是矩阵 A 的特征值， x 是 A 对应于 λ 的特征向量。矩阵 A 的特征值分解就是将 A 分解为以下形式：

$$A = U\Lambda U^{-1}$$

或者，在U为正交矩阵（即U的列向量是单位向量且两两正交）的情况下，由于正交矩阵的逆等于其转置，上式可以写为：

$$A = U\Lambda U^T$$

其中，

- U是由A的特征向量组成的 $n \times n$ 矩阵，其列向量是A的单位化特征向量。
- Λ 是一个 $n \times n$ 的对角矩阵，其对角线上的元素是A的特征值，非对角线上的元素都是0。

注意事项

1. **特征值与特征向量的关系**：特征值是矩阵中与特定特征向量相关的实数，它表示该特征向量在矩阵变换下的缩放比例。特征向量是矩阵变换中方向不变的向量。
2. **矩阵的可对角化**：只有可对角化矩阵（即存在足够数量的线性无关特征向量的矩阵）才能进行特征值分解。
3. **应用领域**：特征值分解在信号处理、统计学、经济学、量子力学等多个领域都有重要应用。例如，在信号处理中，特征值可以用来分析信号的频率成分；在统计学中，主成分分析（PCA）就是基于特征值分解的一种数据降维方法。

在NumPy中，进行特征值分解主要使用 `numpy.linalg.eig` 函数。这个函数计算给定方阵的特征值和右特征向量。对于对称矩阵（或更一般地，Hermitian矩阵），通常还推荐使用 `numpy.linalg.eigh`，因为它在数值上更稳定且效率更高。不过，对于非对称矩阵，应使用 `numpy.linalg.eig`。

以下是如何使用 `numpy.linalg.eig` 进行特征值分解的示例：

```

import numpy as np

# 定义一个方阵A
A = np.array([[4, 2], [1, 3]])

# 使用eig函数进行特征值分解
eigenvalues, eigenvectors = np.linalg.eig(A)

# eigenvalues是一个包含特征值的数组
# eigenvectors的列是对应的特征向量

print("特征值: ", eigenvalues)
print("特征向量: ")
for i, eigenvector in enumerate(eigenvectors.T):
    print(f"特征值 {eigenvalues[i]} 对应的特征向量: {eigenvector}")

# 验证特征值和特征向量的正确性
# 对于每个特征值lambda_i和对应的特征向量v_i, 应有 Av_i = lambda_i * v_i
for i, eigenvalue in enumerate(eigenvalues):
    eigenvector = eigenvectors[:, i]
    result = np.dot(A, eigenvector)
    assert np.allclose(result, eigenvalue * eigenvector)
    print(f"验证特征值 {eigenvalue} 和特征向量 {eigenvector} 正确")

```

接下来我们可以看到两个用了矩阵分解的案例：

案例1：层次分析法

层次分析法是美国运筹学家匹茨堡大学教授萨蒂于20世纪70年代初提出的一种评价策略。这种策略虽然带有一定主观性，但非常奏效，也是在社会科学研究中经常使用的一类方法。

首先，层次分析法的流程分五步走：

1. 选择指标，构建层次模型。
2. 对目标层到准则层之间和准则层到方案层之间构建比较矩阵。
3. 对每个比较矩阵计算CR值检验是否通过CR检验，如果没有通过检验需要调整比较矩阵。
4. 求出每个矩阵最大的特征值对应的归一化权重向量。
5. 根据不同矩阵的归一化权向量计算出不同方案的得分进行比较。

根据上述流程，我们可以亲手写出一个AHP方法的代码：

```

def AHP(A):
    m=len(A)                                #获取指标个数
    n=len(A[0])
    RI=[0, 0, 0.58, 0.90, 1.12, 1.24, 1.32, 1.41, 1.45, 1.49, 1.51]
    R= np.linalg.matrix_rank(A)             #求判断矩阵的秩
    V,D=np.linalg.eig(A)                    #求判断矩阵的特征值和特征向量，V特征值，D特征向量；
    list1 = list(V)
    B= np.max(list1)                         #最大特征值
    index = list1.index(B)
    C = D[:, index]                          #对应特征向量
    CI=(B-n)/(n-1)                           #计算一致性检验指标CI
    CR=CI/RI[n]
    if CR<0.10:
        print("CI=", CI.real)
        print("CR=", CR.real)
        print('对比矩阵A通过一致性检验，各向量权重向量Q为： ')
        sum=np.sum(C)
        Q=C/sum                             #特征向量标准化
        print(Q.real)                        # 输出权重向量
        return Q.real
    else:
        print("对比矩阵A未通过一致性检验，需对对比矩阵A重新构造")
        return 0

```

案例2：主成分分析

主成分分析的主要目的是希望用较少的变量去解释原来资料中的大部分变异，将原始数据中许多相关性较高的变量转化成彼此相互独立或不相关的变量。通常是选出比原始变量个数少，能解释大部分资料中的变异的几个新变量（也就是主成分）。因此，我们可以知道主成分分析的一般目的是：(1)数据的降维；(2)主成分的解释。

主成分分析包括以下几个步骤.

(1) 数据的去中心化：对数据表 X 中每个属性减去这一列的均值。这样做的目的在于消除数据平均水平对它的影响。

$$\bar{x} = (x_1 - \bar{x}_1, x_2 - \bar{x}_2, \dots, x_n - \bar{x}_n)$$

(2) 求协方差矩阵：注意这里需要除(n-1)。

$$C = \frac{1}{n-1} \bar{X}^T \bar{X}$$

(3) 对协方差矩阵进行特征值分解。

$$C = Q \Sigma Q^T$$

(4) 特征值排序：挑选更大的 k 个特征值，将特征向量组成矩阵 P 。

(5) 进行线性变换 $F = PX$ ，从而得到主成分分析后的矩阵。矩阵中的每一列称作一个主成分，选择的特征值经过归一化就变成了权重。

注意：实际过程中PCA的底层做矩阵分解时使用奇异值分解（SVD）更多一些，这里用特征值分解更加容易理解。

我们同样可以写出它的实现代码：

```
def pca(X,n_components):
    X=np.array(X)
    X=X-np.mean(X)
    n=len(X)
    A=np.dot(X.T,X)/(n-1)
    V,D=np.linalg.eig(A)
    idx = (-V).argsort(axis=None)[:n_components]
    P=D[idx]
    F=np.dot(X,P.T)
    return V[idx]/sum(V),F
```

2. 奇异值分解

奇异值分解（Singular Value Decomposition，简称SVD）是线性代数中一种重要的矩阵分解方法，它在多个领域如信号处理、统计学等中都有重要应用。以下是奇异值分解的具体定义：

奇异值分解是指将一个非零的 $m \times n$ 实矩阵 A （其中 $A \in R^{(m \times n)}$ ），表示为以下三个实矩阵乘积形式的运算，即进行矩阵的因子分解：

$$A = U \Sigma V^T$$

其中：

- U 是 $m \times m$ 阶正交矩阵（orthogonal matrix），它的列向量称为左奇异向量。
- Σ 是 $m \times n$ 阶对角矩阵，其对角线上的元素（非负实数）按降序排列，称为奇异值。除了对角线上的元素外， Σ 的其他位置元素均为0。

- V 是 $n \times n$ 阶正交矩阵，它的列向量称为右奇异向量。
- V^T 是 V 的转置矩阵。

它具备如下一些性质

1. **存在性**：任意 $m \times n$ 的矩阵 A 的奇异值分解一定存在，但可能不唯一（特别是当矩阵 A 存在退化的奇异值时）。
2. **最优近似**：奇异值分解可以被视为在平方损失（Frobenius范数）意义下对矩阵的最优近似。在实际应用中，常常通过截断奇异值分解（只取最大的 k 个奇异值）来实现矩阵的近似表示，这在数据压缩等领域尤为重要。
3. **几何意义**：从线性变换的角度来看， $m \times n$ 矩阵 A 表示从 n 维空间到 m 维空间的一个线性变换。奇异值分解可以将这个线性变换分解为三个简单的变换：一个坐标系的旋转或反射变换、一个坐标轴的缩放变换（缩放因子即为奇异值）、另一个坐标系的旋转或反射变换。

奇异值分解在多个领域有广泛应用，包括但不限于：

- **数据压缩**：通过截断奇异值分解，可以实现对原始数据的近似表示，从而达到数据压缩的目的。
- **信号处理**：在信号处理中，奇异值分解被用于分析信号的频率成分和进行信号重建。
- **统计学**：在统计学中，奇异值分解被用于主成分分析（PCA），一种数据降维方法，用于找出数据中的主要模式。

在NumPy中，进行奇异值分解（SVD）主要使用 `numpy.linalg.svd` 函数。这个函数返回给定矩阵的奇异值分解结果，包括奇异值、左奇异向量和右奇异向量。

以下是如何使用 `numpy.linalg.svd` 进行奇异值分解的示例：

```

import numpy as np

# 定义一个矩阵A
A = np.array([[1, 2], [3, 4], [5, 6]])

# 使用svd函数进行奇异值分解
U, s, VT = np.linalg.svd(A, full_matrices=False)

# 注意: numpy.linalg.svd返回的s是奇异值数组, 而不是对角矩阵
# 我们需要将其转换为对角矩阵形式
Sigma = np.zeros((A.shape[0], A.shape[1]))
Sigma[:min(A.shape[0], A.shape[1]), :min(A.shape[0], A.shape[1])] = np.diag(s)

# VT是V的转置, 如果需要V, 则可以使用VT.T获取
V = VT.T

# 验证SVD的正确性
# A应该等于U * Sigma * VT (注意Sigma不是方阵时, 需要按照形状进行乘法)
reconstructed_A = np.dot(U, np.dot(Sigma, VT))

# 由于浮点运算的精度问题, 我们不会直接比较A和reconstructed_A是否相等
# 而是检查它们是否足够接近
print(np.allclose(A, reconstructed_A)) # 应该输出True或非常接近True的值

# 输出奇异值、左奇异向量和右奇异向量
print("奇异值: ", s)
print("左奇异向量: ")
print(U)
print("右奇异向量: ")
print(V)

```

请注意几个关键点:

1. `np.linalg.svd` 的 `full_matrices` 参数控制返回的矩阵U和V的形状。当 `full_matrices=True` 时 (默认值), U和V都是方阵, 且大小分别为 $m \times m$ 和 $n \times n$ (其中m和n是A的行数和列数)。然而, 在许多情况下, 我们不需要完整的U和V矩阵, 只需要包含必要信息的“经济”版本。这可以通过设置 `full_matrices=False` 来实现, 此时U的形状为 $m \times k$, V的形状为 $n \times k$, 其中k是矩阵A的秩 (或更具体地说, 是奇异值的数量)。
2. 返回的 `s` 是一个包含奇异值的数组, 而不是对角矩阵。为了构建对角矩阵 `Sigma`, 我们需要创建一个适当大小的零矩阵, 并将其对角线上的元素设置为 `s` 中的值。

3. 由于浮点运算的精度问题，直接比较两个矩阵是否相等通常不是一个好主意。相反，我们使用 `np.allclose` 来检查两个矩阵是否足够接近。
4. 在实际应用中，SVD的截断版本（即只保留最大的几个奇异值）经常被用于数据压缩、噪声过滤等任务。这可以通过简单地保留 s 、 U 和 V^T （或 V ）中的前 k 个元素来实现。

3. 楚列斯基分解

楚列斯基分解（Cholesky Decomposition）是线性代数中的一个重要概念，特别是在数值分析和计算科学领域中被广泛应用。楚列斯基分解是指将一个对称正定矩阵 A 表示为 LL^T 的形式，其中 L 是一个下三角矩阵，且其对角线元素都是非负实数。这里， L^T 表示 L 的转置矩阵。这种分解使得我们能够高效地解决线性系统 $Ax = b$ ，而无需直接处理矩阵 A 。

1. **对称正定矩阵**：楚列斯基分解仅适用于对称正定矩阵。对称矩阵意味着 $A = A^T$ ，即矩阵的转置等于其自身。正定矩阵则意味着对于任何非零向量 x ，都有 $x^T Ax > 0$ 。
2. **下三角矩阵**：分解得到的 L 是一个下三角矩阵，即除了对角线及其下方的元素外，其他元素都为 0。
3. **非负对角线元素**： L 的对角线元素都是非负的，这是为了确保 LL^T 的结果是一个正定矩阵。
4. **高效性**：楚列斯基分解在计算上非常高效，特别是对于大型稀疏对称正定矩阵。它允许我们以较低的计算和存储成本解决线性系统。

注意事项

- 在进行楚列斯基分解之前，通常需要检查矩阵是否是对称正定的。不是所有对称矩阵都能进行楚列斯基分解。
- 对于某些特殊类型的对称正定矩阵（如希尔伯特矩阵），可能需要特别的算法来确保计算的稳定性和准确性。

在NumPy中，楚列斯基分解（Cholesky Decomposition）可以通过 `numpy.linalg.cholesky` 函数直接实现。这个函数接受一个对称正定矩阵作为输入，并返回其下三角的Cholesky因子（即 L 矩阵），使得 $L @ L.T$ （ $@$ 是矩阵乘法运算符）等于原始矩阵。

以下是一个使用NumPy实现楚列斯基分解的示例代码：

```

import numpy as np

# 定义一个对称正定矩阵A
A = np.array([[4, 12, -16],
              [12, 37, -43],
              [-16, -43, 98]])

# 检查A是否是对称矩阵（可选步骤，但进行Cholesky分解前通常需要确保这一点）
assert (A == A.T).all(), "Matrix A is not symmetric."

# 使用numpy.linalg.cholesky进行楚列斯基分解
L = np.linalg.cholesky(A)

# 验证分解结果: L @ L.T 应该等于 A
assert np.allclose(L @ L.T, A), "Cholesky decomposition failed."

# 输出L矩阵
print("L矩阵（Cholesky因子）:")
print(L)

```

在这个例子中，我们首先定义了一个对称正定矩阵 A 。然后，我们使用 `numpy.linalg.cholesky` 函数来计算其Cholesky因子 L 。之后，我们通过检查 $L @ L.T$ 是否等于原始矩阵 A 来验证分解的正确性。最后，我们输出了 L 矩阵。

请注意，在实际应用中，你可能不需要显式地检查矩阵是否对称，因为如果你确定你的矩阵是对称正定的，那么就可以直接使用 `numpy.linalg.cholesky` 函数。然而，如果矩阵可能不是对称的，或者你不确定它是否正定，那么在调用 `numpy.linalg.cholesky` 之前进行检查是一个好习惯。

另外，需要注意的是，`numpy.linalg.cholesky` 函数只返回下三角的Cholesky因子 L ，而不返回 L 的转置或完整的分解形式 $L L^T$ 。但是，由于 L 是下三角的，你可以很容易地通过计算 $L @ L.T$ 来得到原始矩阵 A 。

4. QR分解

QR分解（QR Decomposition），也被称为正交三角分解法，是一种将矩阵分解为一个正交矩阵 Q 和一个上三角矩阵 R 的乘积的矩阵分解方法。对于任意实（或复）矩阵 A ，如果它能够被分解为一个正交（或酉）矩阵 Q 和一个上三角矩阵 R 的乘积，即 $A = QR$ ，则称这种分解为 A 的QR分解。

- **正交矩阵 Q** : Q 是一个 $m \times m$ 的正交矩阵（对于复数矩阵，则是酉矩阵），满足 $Q^T * Q = I$ （或 $Q^H * Q = I$ ，其中 H 表示共轭转置， I 是单位矩阵）。正交矩阵的列向量是单位向量且两两正交。

- **上三角矩阵R**: R是一个 $m \times n$ 的上三角矩阵, 其对角线以下的元素全为0 (对于方阵, 则是一个上三角矩阵; 对于非方阵, R的右侧可能会有一些行是全0的)。
- **适用性广**: QR分解适用于对称矩阵和非对称矩阵, 也适用于实矩阵和复矩阵。
- **计算特征值**: QR分解是求一般矩阵全部特征值的最有效并广泛应用的方法之一。一般矩阵先经过正交相似变化成为Hessenberg矩阵, 然后再应用QR方法求特征值和特征向量。
- **线性最小二乘**: QR分解被广泛应用于线性最小二乘问题的求解, 通过将问题转化为上三角矩阵的求解, 可以简化计算过程。

QR分解有多种算法实现, 包括但不限于Gram-Schmidt算法、Householder算法和Givens算法。这些算法各有特点, 但目标都是将矩阵A分解为一个正交矩阵Q和一个上三角矩阵R的乘积。

- **Gram-Schmidt算法**: 通过一系列的投影操作, 将A的列向量转化为正交向量, 从而构造出Q和R。
- **Householder算法**: 利用Householder反射矩阵, 通过一系列的反射操作, 将A转化为上三角矩阵R, 同时构造出正交矩阵Q。
- **Givens算法**: 通过一系列的旋转操作, 逐步将A的下三角元素化为0, 同时保持上三角元素不变, 从而得到R, 并构造出Q。

在NumPy中, QR分解可以通过 `numpy.linalg.qr` 函数直接实现, 而不需要手动编写QR分解的算法。 `numpy.linalg.qr` 函数内部已经实现了这些算法中的一种 (通常是Householder算法, 因为它在数值上更稳定), 并为用户提供了方便的接口。

以下是使用NumPy进行QR分解的示例代码:

```

import numpy as np

# 定义一个矩阵A
A = np.array([[12, -51, 4],
              [6, 167, -68],
              [-4, 24, -41]])

# 使用numpy.linalg.qr进行QR分解
Q, R = np.linalg.qr(A)

# Q是一个正交矩阵（或酉矩阵，对于复数矩阵），R是一个上三角矩阵
# 验证QR分解：Q @ R 应该等于原始矩阵A（由于浮点运算的精度问题，使用np.allclose进行比较）
assert np.allclose(Q @ R, A), "QR decomposition failed."

# 输出Q和R
print("Q矩阵（正交矩阵）:")
print(Q)
print("R矩阵（上三角矩阵）:")
print(R)

```

在这个例子中，`numpy.linalg.qr` 函数返回了两个矩阵：Q 和 R。Q 是一个正交矩阵（其列向量是单位向量且两两正交），而 R 是一个上三角矩阵。这两个矩阵的乘积 $Q @ R$ 应该等于原始矩阵 A（由于浮点运算的精度问题，我们通常不会直接比较它们是否完全相等，而是使用 `np.allclose` 来检查它们是否足够接近）。

需要注意的是，`numpy.linalg.qr` 函数还可以接受一个 `mode` 参数，该参数控制函数的输出。默认情况下（`mode='reduced'`），Q 的形状是 (m, k) ，其中 m 是 A 的行数， k 是 A 的列数（如果 A 是方阵，则 k 等于 m ；如果 A 是瘦矩阵，则 k 小于 m ），而 R 的形状是 (k, n) ，其中 n 是 A 的列数。如果 `mode='complete'`，则 Q 的形状是 (m, m) ，并且 R 的底部会包含一些额外的行（对于非方阵 A，这些行将是全零的），以便 Q 成为一个完整的正交矩阵。然而，在大多数情况下，默认的 `mode='reduced'` 就足够了。